



UNIVERSITAT POLITÈCNICA DE CATALUNYA
BARCELONATECH
Centre de la Imatge i la Tecnologia Multimèdia

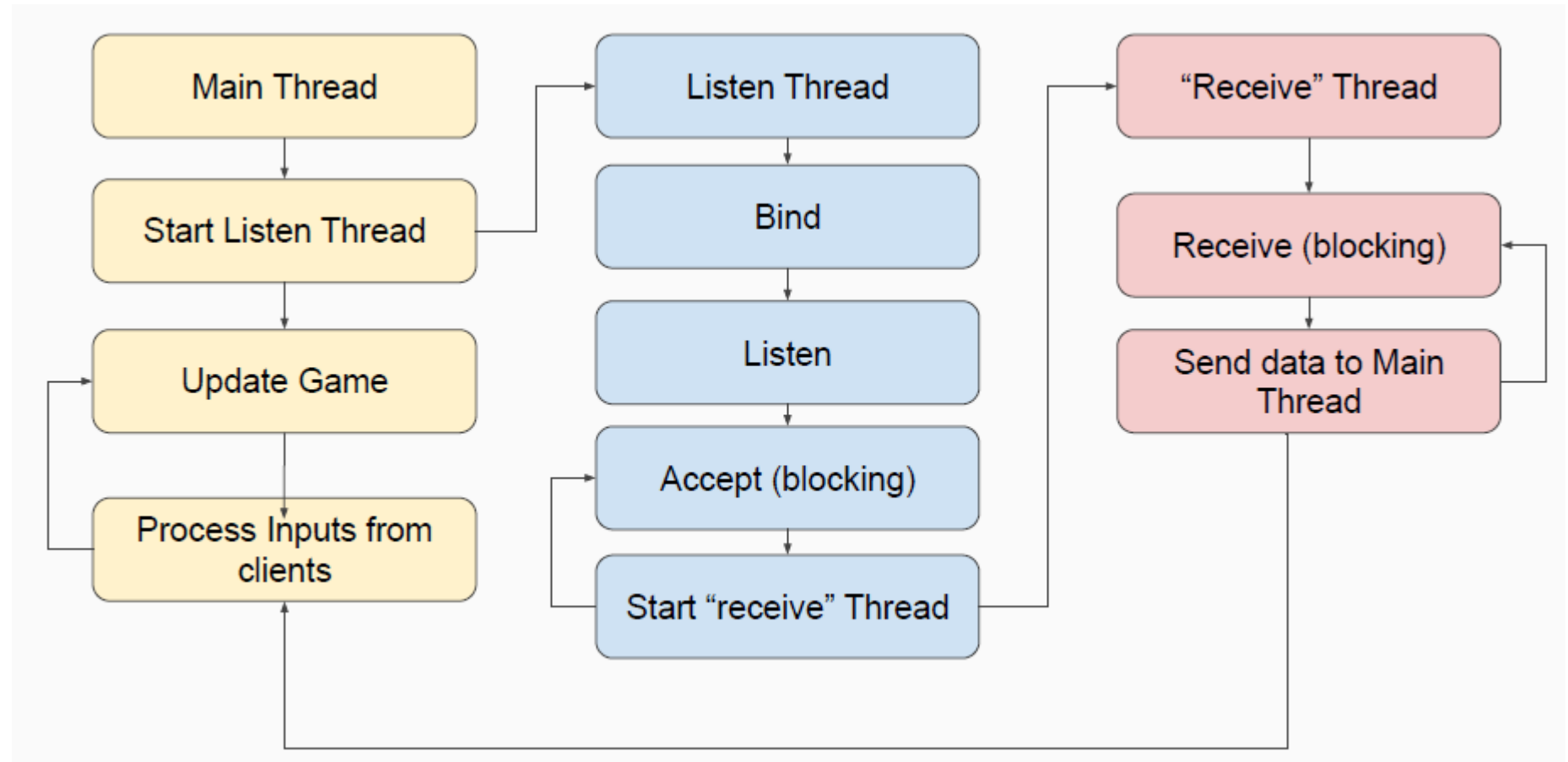
Lab Session 2.2:TCP / UDP



Blocking Operations

- Sending or receiving data
- Listening for connections
- Trying to connect
- ...
- Avoid affecting framerate!

Option 1: Multithreading



Option 1: Multithreading

- Scalability?
- How many threads are required for 5 clients?
- It's manageable solution for handling a few sockets. But can become tricky.

Option 2: Non-blocking Methods

- Begin Accept
 - Begin Connect
 - Begin Receive
 - + callbacks
 - ...
-
- Pro: Non-blocking. Throw an event and call a Callback function when “done” for asynchronous handling
 - Con: Still uses threads behind. Similar scalability problems when too many connections
 - Use only with very few sockets (clients, not servers)

Option 2: Non-blocking methods

```
void startReceive()
{
    StateObject state = new StateObject();
    client.BeginReceive( data, 0, BuffSize, 0, new AsyncCallback(ReceiveCallback), state);
}

void ReceiveCallback( IAsyncResult ar )
{
    StateObject state = (StateObject) ar.AsyncState;
    Socket client = state.workSocket;
    // Read the data:
    int bytesRead = client.EndReceive(ar);
    Debug.Log(Encoding.ASCII.GetString(state.buffer,0,bytesRead));
    // Usually, keep reading because there might be more data to read
    client.BeginReceive(state.buffer,0,StateObject.BufferSize,0, new AsyncCallback(ReceiveCallback), state);
}
```

Option 3: Select() function

```
public static void Select (System.Collections.IList? checkRead, System.Collections.IList? checkWrite,  
                          System.Collections.IList? checkError, int timeout);
```

- Consults a list of sockets to see what operation can be called on them without blocking
- checkRead, checkWrite and checkError will return the same list with the "non-ready" elements removed
 - e.g. waiting for data on 5 sockets (clients) and 2 already sent a message
- First, checkRead will have length of 5 (all the sockets you have active)
- Select will change it to length 2 (only the sockets where you can read without blocking)

<https://learn.microsoft.com/en-us/dotnet/api/system.net.sockets.socket.select?view=net-5.0>

Deliverable 2: Sockets

Exercise 2

- On your game create 2 scenes:
 - Server scene:
 - Probably named “create game” or similar
 - It starts a socket and waits for incoming messages from the client scene
 - It answers with the name of the server
 - Client scene:
 - Probably named “join game”
 - It allows to insert an IP
 - It connects to a server, knowing it’s IP
 - It sends a message with the user’s name
 - Both scenes:
 - Bonus: Lead to a waiting room
 - Bonus: Extend the waiting room with extra information and message exchanges in a chat
 - Bonus: Accept several connections
- Implement both cases using UDP and using TCP
- Extend it as you like

Submissin rules

- .ZIP file
 - README: Txt/pdf with your improvements, instructions, any info you think should be known beforehand
 - .Exe
 - Small video showing the functionality
 - A package of all the relevant scenes
- Name it Surname_Name_Lab2